

Angèle Mounier
B2



Lien vers le repo Github

Sommaire

1. Objectif
2. Mon projet
3. Structure de ma page
4. Techniques utilisées
5. Arborescence du code
6. Explication détaillée de mon code

Objectif

L'objectif de ce projet est de présenter la météo d'un endroit dans une page web d'un office de tourisme. Pour cela, la page doit accéder à une API afin d'afficher la météo d'un point géographique.

Contraintes techniques

Le projet doit être réalisé en JS modulaire grâce au bundler Webpack. Il doit interroger l'API Open Météo pour obtenir les prévisions météorologiques d'un point donné. Pour appeler cette API, il faut utiliser la bibliothèque JavaScript Axios.



webpack

Webpack est un bundler permettant de créer un fichier bundle.js où sont rassemblés tous les fichiers.js du projet.

Mon projet



J'ai choisi d'incarner l'office de tourisme des Herbiers. Les Herbiers est une commune située dans le département de la Vendée (85), à proximité du Puy du Fou.

Logo



version claire

Couleurs



Typographie

Nunito Sans

ABCDEFGHIJKLMNOPQRSTUVWXYZ
abcdefghijklmnopqrstuvwxyz

Angèle Mounier

Structure de ma page

J'ai choisi de construire ma page en 4 zones : en-tête, météo, agenda, activités

1 En-tête : cette zone comprend un menu et deux boutons: nous contacter et voir la météo



2 Météo : cette zone affiche la météo des 7 jours à venir sous formes de cartes

Météo des Herbiers

Météo des 7 jours à venir

ven. 31 oct.



17° / 12.1°

sam. 1 nov.



16.4° / 11.8°

dim. 2 nov.



13.4° / 7.9°

lun. 3 nov.



15.9° / 7.7°

[Voir la météo complète des Herbiers](#)

3 Agenda : cette zone affiche les évènements à venir



4 Activités : cette zone affiche les activités à faire dans la région



Angèle Mounier

Techniques utilisées



JS modulaire

Comme demandé dans le sujet, j'ai réalisé mon projet en JS modulaire. Cette technique consiste à créer plusieurs fichiers JS. Cela permet de trier les informations et facilite le codage : on sait où retrouver les informations à modifier. Le JS modulaire donne une structure claire au projet puisque chaque fichier.js est responsable d'une fonction.

Tous les fichiers doivent donc apparaître sur le fichier.html? Non! En effet, le bundler Webpack permet de réaliser, avec tous nos fichiers.js et .css, un seul fichier "bundle.js", qui, lui, sera mentionné dans le fichier index.html grâce à la balise `<script>`.

JS asynchrone

Le JS asynchrone permet de ne pas bloquer l'exécution du code. En effet, en JS synchrone, il faut attendre que l'instruction précédente soit réalisée avant de passer à l'instruction suivante. Si la première instruction comporte une erreur, cela bloque toutes les instructions suivantes. Le JS asynchrone, lui, permet de ne pas bloquer toutes les instructions si l'une d'elles est défectueuse. J'ai utilisé le JS asynchrone pour l'appel de l'API. Ainsi, quand mon code concernant l'appel de l'API ne fonctionnait pas ou mettait du temps à afficher la météo, le reste de la page s'affichait.

Angèle Mounier

Arborescence du code

Dossier dist avec

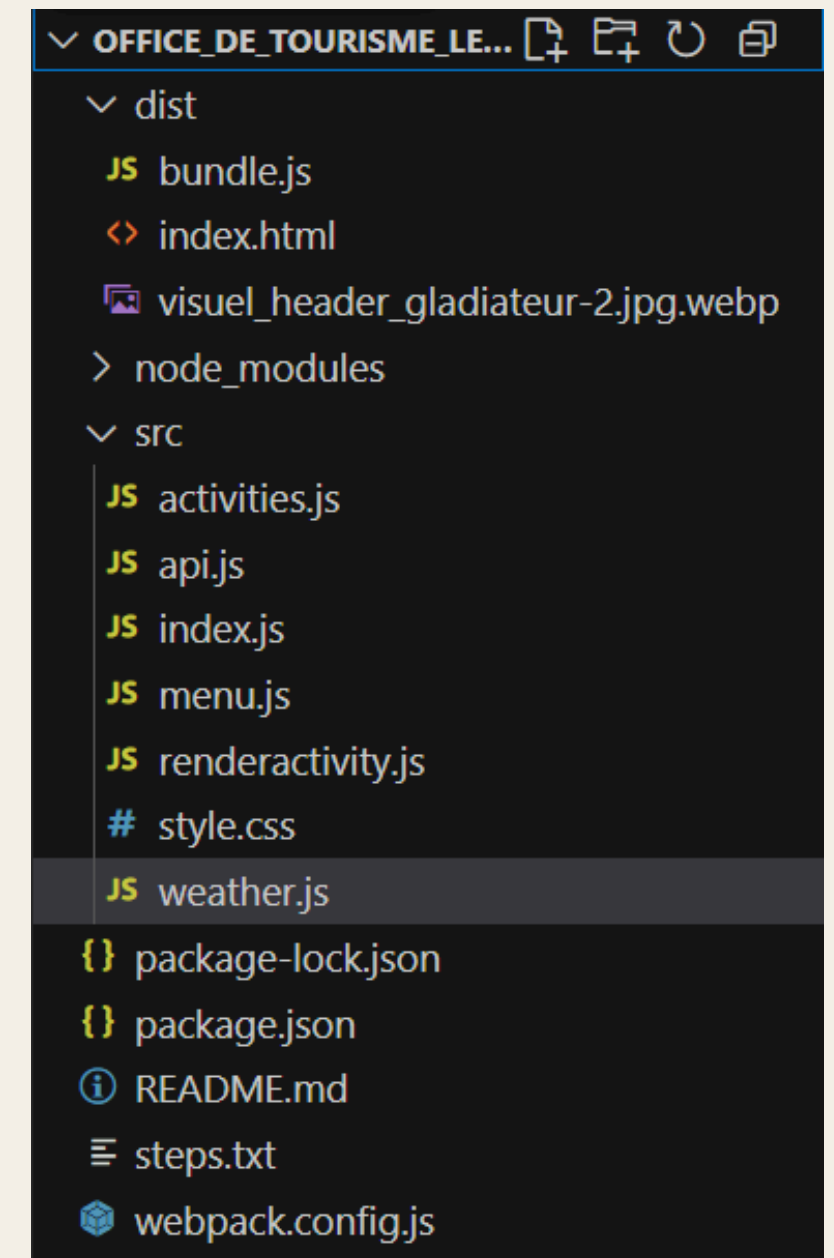
- le fichier bundle.js construit par webpack
- le fichier index.html avec le contenu de ma page et la balise <script> qui charge le bundle.js
- le visuel d'en-tête chargé directement par le fichier html

Dossier node_modules téléchargé lors de l'installation d'Axios

Dossier src avec tous les fichiers qui seront pris en compte par webpack lors de la création ou de la mise à jour du bundle.js

- le fichier activities.js qui répertorie les informations des futures cartes de la section activité
- le fichier api.js permettant de requêter l'API
- le fichier index.js qui est le fichier entrée utilisé par webpack. On y importe les fonctions des différents fichiers. Webpack se basera sur ce fichier pour construire le fichier final bundle.js
- le fichier menu.js qui charge le menu
- le fichier renderactivity.js qui crée et charge les cartes "activités"
- le fichier style.css permettant de bien présenter la page
- le fichier weather.js qui affiche les résultats de la sollicitation de l'API sous forme de cartes

Le fichier webpack.config.js permet de désigner le fichier index.js comme le fichier entrée à partir duquel webpack pourra construire le bundle.js



1. Installer webpack et la bibliothèque axios
2. Appel de l'API
3. Mise en forme des résultats de l'appel de l'API
4. Menu
5. Zone agenda
6. Zone activités
7. Outils

Webpack et Axios

Installation de Webpack et d'Axios via un Command Prompt dans le terminal VS Code

1. commande npm init : création d'un fichier package.json
2. commande npm install webpack webpack-cli : création d'un dossier node_modules et d'un fichier package-lock.json
3. commande npm install axios : installation de la bibliothèque Axios permettant d'interroger une API

Création du bundle.js par Webpack

Pour créer le fichier bundle.js, qui rassemblera tout notre JS, il faut ensuite créer dans un dossier src un fichier index.js, où sont importées toutes les fonctions des fichiers.js présent dans le dossier src.

On configure ensuite le fichier webpack.config.js

Après chaque édition, la commande npx webpack reconstruit le fichier bundle.js.

```
const path = require('path');

module.exports = {
  entry: './src/index.js',
  output: {
    filename: 'bundle.js',
    path: path.resolve(__dirname, 'dist'),
  },
  module: {
    rules: [
      {
        test: /\.css$/i, // tous les fichiers .css
        use: ['style-loader', 'css-loader'],
      },
    ],
  },
  mode: 'production',
};
```

Fichier webpack.config.js

Il définit une entrée : index.js, ainsi que le nom du fichier construit (ici : bundle.js). Il permet également d'intégrer du CSS dans le bundle.js

Appel de l'API

L'appel de l'API se fait via le fichier api.js

Il s'agit de l'API Open Météo. On l'appelle grâce à Axios. L'URL est définie par ce que l'on veut obtenir. Ici on veut obtenir la météo pour les Herbiers grâce à ses coordonnées géographiques (latitude=46.871&longitude=-1.016), avec, pour chaque jour, les températures minimum et maximum (daily=temperature_2m_max,temperature_2m_min) ainsi que le weathercode qui permettra ensuite de traduire en pictogrammes le temps qu'il fait (pluvieux, nuageux, ensoleillé,...)

Fonction asynchrone

L'appel de l'API se fait grâce à la fonction asynchrone "getWeatherData". "Await" permet d'attendre la réponse d'axios avant de retourner la réponse, c'est à dire de charger les données.

```
import axios from 'axios';

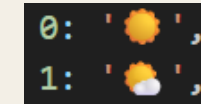
export async function getWeatherData() {
  const url = 'https://api.open-meteo.com/v1/forecast?latitude=46.871&longitude=-1.016';

  try {
    const response = await axios.get(url);
    return response.data;
  } catch (error) {
    console.error('Erreur lors de la récupération des données météo :', error);
    return null;
  }
}
```

Mise en forme

La mise en forme des résultats de l'appel de l'API se fait grâce au fichier `weather.js`

Ce fichier permet tout d'abord de définir un pictogramme pour chaque weathercode renvoyé par l'API. Ex:



Ensuite il définit une fonction `formatDate` permettant de donner un format à la date. Cette fonction est utilisée ensuite dans la fonction suivante.

Elle n'est donc pas exportée.

Enfin vient la fonction qui sera ensuite exportée intitulée `display7DayForecast`. Celle-ci permet de récupérer les informations données par l'API comme la température minimum, la température maximum ou le weathercode et de les afficher sous forme de cartes. Pour cela, la fonction utilise une boucle `for`, qui, pour chacun des 7 jours à venir, construit une div html qui affichera les données.

```
const dates = data.daily.time;  
const tempMax = data.daily.temperature_2m_max;  
const tempMin = data.daily.temperature_2m_min;  
const codes = data.daily.weathercode;
```

récupération des données

```
for (let i = 0; i < 7; i++) {  
  const date = formatDate(dates[i]);  
  const icon = weatherCodeToIcon[codes[i]] || '?';  
  html += `  
    <div class="weather-day">  
      <div class="day">${date}</div>  
      <div class="icon">${icon}</div>  
      <div class="temps">  
        <span class="max">${tempMax[i]}°</span> /  
        <span class="min">${tempMin[i]}°</span>  
      </div>  
    </div>  
  `;  
}
```

affichage des données dans des
éléments html grâce à une boucle `for`

Menu

Création du menu grâce à une fonction “create Menu” dans le fichier menu.js

Ce menu devait initialement mener vers d'autres pages HTML.

La fonction createMenu est exportée vers le fichier index.js. Dans le fichier index.js, cette même fonction est importée depuis le fichier menu.js

```
export function createMenu() {  
  const navbar = document.getElementById('navbar');  
  navbar.innerHTML = `  
    <nav class="navbar">  
      <ul class="nav-links">  
        <li><a href="index.html">Accueil</a></li>  
        <li><a href="activites.html">Activités</a></li>  
        <li><a href="agenda.html">Agenda</a></li>  
        <li><a href="logements.html">Logements</a></li>  
        <li><a href="contact.html">Contact</a></li>  
      </ul>  
    </nav>  
  `;  
  document.body.prepend(header);  
}
```

menu.js

```
import { createMenu } from './menu';
```

index.js

Zone agenda

La zone agenda n'est pas gérée par du JS. Elle est entièrement codée en HTML. Dans un conteneur 'event-contener', se trouvent trois div de classe 'card'. Ces trois div sont les trois cartes événements qui constitue la zone agenda.

```
<div class="card">
  
  <div class="card-body">
    <h5 class="card-title">Concert des trois chorales des Herbiers</h5>
    <p class="card-text">Les trois chorales des Herbiers La Rose d'Or, Amis Voix et Donjon d'Ardelay, se retrouvent pour un co
    <h6 class="dates">📅 30 octobre</h6>
  </div>
</div>
```

exemple de carte



Angèle Mounier

Zone activités

La zone activités est gérée par deux fichiers JS : activity.js et renderactivity.js.

Le fichier **activity.js** permet de stocker dans la fonction activity les informations de chacune des activités: nom, description, image et catégorie (pour le filtrage). Cette fonction, exportée, est importée par le fichier renderactivity.js qui récupère les informations des cartes.

```
{
  id: 1,
  name: "Circuit de promenade des Alouettes",
  description: "Circuit pédestre accessible aux familles",
  image: "https://www.lesherbiers.fr/media/2000-* -999",
  category: "out"
},
```

activity.js

```
export const activity = [
```

export de la fonction
activity depuis le
fichier activity.js

import de la fonction dans renderactivity.js

```
import { activity } from "../activities.js";
```

Le fichier **renderactivity.js** permet de récupérer les informations et les affiche en cartes grâce à la fonction stocker dans la fonction renderActivity. Celle-ci construit les cartes en fonction du filtre présent dans le fichier HTML.

```
filteredActivity.forEach(activity => {
  const card = document.createElement("div");
  card.className = "card cardActivity";

  const img = document.createElement("img");
  img.src = activity.image;
  card.appendChild(img);

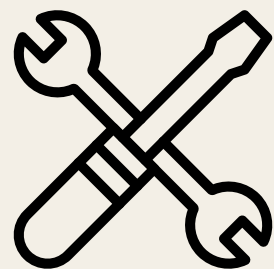
  const title = document.createElement("h5");
  title.textContent = activity.name;

  const description = document.createElement("p");
  description.textContent = activity.description;

  const info = document.createElement("div");
  info.className = "card-body";
  info.appendChild(title);
  info.appendChild(description);

  card.appendChild(info);
  activityContainer.appendChild(card);
});
```

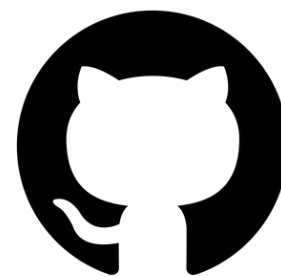
construction des cartes
avec les éléments de la
fonction activity.



Outils



GitKraken



GitHub



webpack



Merci
pour votre attention